

1. Языки программирования высокого уровня

Что это такое. Язык высокого уровня (ВУЯП) — это язык, дающий программисту удобную абстракцию от аппаратных деталей (регистров, адресов памяти, манипуляций с байтами). Цель — повысить читаемость, ускорить разработку и уменьшить вероятность ошибок.

Ключевые признаки:

- Высокая степень абстракции (переменные, функции, объекты, модули).
- Богатая стандартная библиотека и средства работы с I/O, строками, коллекциями.
- Автоматизация (автоматическое управление памятью/GC в некоторых языках).
- Портруемость: код обычно не зависит от конкретной архитектуры.
- Статическая или динамическая типизация; сильная/слабая типизация.
- Мост между интерпретируемыми, байткодными (VM) и компилируемыми языками.

Категории вУЯП по исполнению:

- Компилируемые в нативный код (C, C++ — частично низкоуровневые/«среднего уровня»).
- Компилируемые в байткод с VM и JIT (Java, C#).
- Интерпретируемые/VM (Python, Ruby — CPython/CRuby исполняют байткод).
- JIT-компилируемые (современный JS-движок — V8).
- Смешанные (Go, Rust — компилируемые, но проектируются как «высокого уровня» с системным контролем).

Компиляция в нативный код

- **Что это:** исходный код → машинные инструкции конкретного процессора (x86, ARM и т.д.).
- **Ключевая особенность:** код уже готов для выполнения конкретной архитектурой.
- **Плюсы:** быстрое выполнение (нет доп. шагов на запуск).
- **Минусы:** нужно перекомпилировать под каждую архитектуру.

- **Пример:** C, C++, Rust (генерируют .exe или ELF, завязанный на архитектуру).

Компиляция в байткод

- **Что это:** исходный код → **байткод** (абстрактные инструкции для виртуальной машины, а не конкретного процессора).
- **Плюсы:**
 - Один байткод можно запустить на любой платформе, где есть соответствующая VM (Java → JVM, C# → CLR).
 - Легче портировать.
- **Минусы:** байткод **не** выполняется напрямую процессором — нужна виртуальная машина или JIT.
- **Пример:** Java (.class → JVM), C# (.dll → CLR).

JIT (Just-In-Time compilation)

- **Что это:** динамическая компиляция **во время выполнения программы**.
- Байткод сначала интерпретируется, а **часто используемые участки** компилируются в нативный код прямо во время работы программы.
- **Зачем:** ускоряет часто повторяющийся код, сохраняя портируемость (байткод — один на всех).
- **Плюсы:** сочетает гибкость байткода и скорость нативного кода, может оптимизировать под конкретное железо.
- **Минусы:** задержка на старте (нужно компилировать в процессе).

Плюсы/минусы:

- **Плюсы:** скорость разработки, читаемость, безопасность (в языках с GC/строгой типизацией), богатая экосистема.
- **Минусы:** потенциально меньшая скорость/контроль по сравнению с «чисто» низкоуровневыми средствами; в некоторых случаях — непредсказуемое поведение (GC паузы, автоприведение типов).

2. Целочисленные типы данных в языках высокого уровня

Определение. Целочисленный тип — тип для представления целых чисел (без дробной части). Представляются в двоичном виде внутри машины; семантика и размер задаются языком/реализацией.

Как числа хранятся (кодирования):

- **Двоичное представление** — базовое.
- **Знаковые представления:**
 - **Двоичное дополнительное (two's complement)** — наиболее распространено. Позволяет использовать одну операцию сложения и одинаковую арифметику для знаковых/беззнаковых.
 - **Знак-модуль и ones' complement** встречаются редко.
- **Беззнаковые (unsigned)** — только неотрицательные, диапазон смещается в положительную часть.

2.1. Беззнаковые числа (unsigned)

- **Принцип:** все биты представляют само число.
- Для n бит:
 - Диапазон: $0 \dots 2^n - 1$.
- **Пример (8 бит):**

```
python-repl
КопироватьРедактировать
00000000 = 0
00000001 = 1
...
11111111 = 255
```

2.2. Знаковые числа

Тут нужны способы хранить и положительные, и отрицательные.
Сейчас везде используют **two's complement** (дополнительный код).

Почему его любят:

- Упрощает схемы сложения/вычитания в процессоре.
- Нет двух нулей (+0 и -0), как в старых схемах.

Two's complement (дополнительный код)

- Старший бит (MSB) — знак:
 - 0 → число положительное.
 - 1 → число отрицательное.
- Отрицательные числа получают так: **берём двоичную запись положительного, инвертируем биты и прибавляем 1.**

Пример (8 бит):

diff

КопироватьРедактировать

+5 = 00000101

-5:

1) Инверсия: 11111010

2) +1: 11111011

- Диапазон для n бит: $-2^{(n-1)} \dots 2^{(n-1)}-1$.
 - Для 8 бит: -128 ... 127.

Диапазоны (формула): для n бит:

- беззнаковый: $0 \dots 2^n - 1$
- знаковый (two's complement): $-2^{(n-1)} \dots 2^{(n-1)}-1$
 — пример: n = 8 → signed: -128 ... 127, unsigned: 0 ... 255.

Типичные названия и размеры (общепринятые):

- C/C++: char (1 байт), short (≥16 бит), int (обычно 32 бита), long (32/64 в зависимости от платформы), long long (≥64). Поведение частично зависит от платформы и стандарта.
- Java: byte 8, short 16, int 32, long 64 (всегда знаковые, two's complement).
- Python: int — **произвольной точности** (бесконечная точность в рамках памяти).

- JavaScript: исторически Number (IEEE-754 double — целые безопасно до $\pm(2^{53}-1)$); есть BigInt для произвольных целых.
- Rust/Go/C#: имеют наборы фиксированных размеров (i8, i16, i32, i64, i128, isize/usize и т.д.).

3. Операции над целыми типами в языках высокого уровня

Разобьём операции на группы и перечислим важные моменты и примеры.

3.1 Арифметические операции

- +, -, *, /, % (остаток).
- Унарные +, -.
- ++, -- (в C/Java и др. — префикс/постфикс, в Python их нет).

Особенности деления и остатка:

- Семантика деления/остатка с отрицательными операндами различается между языками.
 - C/C++ (с C99): целочисленное деление усечения (truncation toward zero): $-3 / 2 == -1$, $-3 \% 2 == -1$.
 - Java: поведение аналогично (усечение к нулю).
 - Python: / даёт float, // — целочисленное (floor division): $-3 // 2 == -2$. % в Python даёт результат с знаком делителя: $-3 \% 2 == 1$.
- Внимательно учите семантику на экзамене — разница trunc vs floor частая ловушка.

Переполнение:

- В языках с фиксированной шириной (C, Java, C# без checked) — переполнение ведёт к wrap-around (или UB для signed в C).
- В Python/BigInt — переполнения нет.

3.2 Побитовые операции

- AND &, OR |, XOR ^, NOT ~.
- Сдвиги: << (влево), >> (вправо — арифметический/логический в зависимости от языка).

- Java: `>>` — арифметический (сохраняет знак), `>>>` — логический (вставляет нули).
- В C поведение сдвига отрицательных значений и сдвига сдвига `>=` ширина типа — **неопределено** или `implementation-defined`.
- Битовые операции часто используются для масок, флагов, вырезания битов, извлечения полей.

3.3 Логические vs побитовые операторы

- `&&`, `||` (логические, короткое замыкание) — работают с `boolean`.
- `&`, `|` (побитовые) — если применяются к булевому контексту, не имеют короткого замыкания (в Java/C#).
- Важно не путать: `if ((a & b) != 0)` — проверка битовой маски; `if (a && b)` — логическое.

4. Вещественные типы данных (floating-point) — что это и как устроены

Коротко: вещественный тип — способ представлять числа с дробной частью в двоичной форме. В большинстве современных языков (и в JS) это формат IEEE-754.

Основные понятия (IEEE-754, «binary floating point»)

- **Формат = (знак) + (экспонента) + (мантисса/дробная часть).**
Для формата `binary64` (double, 64 бита): 1 бит знака, 11 битов экспоненты, 52 бита мантиссы (явно хранится 52 бита, фактически 53 значимого бита — благодаря неявной ведущей единице для нормализованных чисел).
Для `binary32` (float, 32 бита): 1 + 8 + 23.
- В контексте вещественных чисел с плавающей точкой (IEEE 754) **мантисса** — это часть числа, которая хранит его значащие цифры, нормализованную так, чтобы перед запятой (в двоичном виде) была 1.
- **Экспонента хранится со смещением (bias).** Для double `bias = 1023`.
Истинная степень = (закодированная экспонента) – `bias`.
- **Нормализованные числа:** ведущий бит мантиссы подразумевается 1 (экономия бита).

- **Subnormal (denormal) числа:** очень близко к нулю, не имеют ведущей единицы — покрывают интервалы между 0 и минимальной нормализованной величиной.
- **Специальные значения:** $+0$ и -0 (разные знаки нуля), $+\infty$, $-\infty$ и NaN (Not a Number — нечисло).
- **Ограничения:** представимы **только** числа, выражаемые как $\text{mantissa} \times 2^{\text{exponent}}$ с ограниченной точностью — многие десятичные дроби (0.1) не представимы точно в двоичной форме.

Почему это важно на практике

- **Погрешности округления** — операции неассоциативны: $(a + b) + c$ может отличаться от $a + (b + c)$.
- **Сравнения на равенство** часто неверны (нужно сравнивать с допуском/эпсилон).
- **Signed zero** влияет на операции вроде $1/0 \rightarrow \text{Infinity}$, $1/-0 \rightarrow -\text{Infinity}$. Некоторые функции (min/max) учитывают знак нуля по-особенному.
- **NaN не равен самому себе** ($\text{NaN} !== \text{NaN}$); чтобы проверить NaN в JS — `Number.isNaN(x)` (или `isNaN` — с оговорками).

Типы и примеры по языкам

- **JavaScript:** Number — IEEE-754 double (binary64). Есть BigInt для целых произвольной длины. Для вычислений с низкой точностью можно использовать Float32Array (typed arrays) — но арифметика в JS всё равно работает через Number, если не работать на уровне бинарных представлений/wasm.
- **C/C++/Java/C#:** float (binary32), double (binary64), плюс long double / extended formats в некоторых реализациях.
- **Языки с десятичной арифметикой:** существуют decimal типы (C# decimal, Java BigDecimal, Python decimal) — полезны для финансов, где важна точность десятичных дробей.

Математические константы и машинная точность

- **Machine epsilon** — минимальное расстояние между 1.0 и следующим представимым числом. Для binary64 это $2^{-52} \approx 2.220446049250313 \times 10^{-16}$. В JS доступно как `Number.EPSILON`.

5. Операции над вещественными типами (особенно в JavaScript)

Базовые операции

- `+`, `-`, `*`, `/`, унарный `-`.
- `%` в JS — остаток (поведение с отрицательными аргументами: результат имеет знак делимого; отличия от `fmod/remainder` в C могут быть).
- `Math`-функции: `Math.abs`, `Math.sqrt`, `Math.pow`, `Math.exp`, `Math.log`, `Math.sin` и т.д. (точность и поведение зависят от реализации движка/платформы).

Специальные аспекты IEEE-754 в операциях

- **NaN**: в большинстве операций, где участвует NaN, результат — NaN (передаётся дальше).
- **Infinites**: арифметика с бесконечностями подчиняется правилам IEEE (например, `Inf + finite = Inf`, `Inf - Inf = NaN`).
- **Signed zero**: `0 === -0` (строгое сравнение), но `Object.is(0, -0)` вернёт `false` в JS. Деление на 0 даёт `±Infinity` в зависимости от знака.
- **Round-to-nearest-even** — стандартное режим округления IEEE (большинство hardware/VM используют именно его по умолчанию), что уменьшает систематическую погрешность при последовательных округлениях.

Infinity и ±0 в JavaScript

В JavaScript числа хранятся в формате **IEEE 754 double precision** (64 бита).

Из-за этого у нас есть:

- **Infinity** — математическая бесконечность.
 - Получить:

js

КопироватьРедактировать

```
let inf = Infinity;           // Позитивная бесконечность
```

```
let negInf = -Infinity;      // Негативная бесконечность
```



```
let divByZero = 1 / 0;    // Infinity
```

- Это особое значение: `typeof Infinity === "number"`
- **+0 и -0** — это два разных значения, но при сравнении считаются равными:

```
js
КопироватьРедактировать
+0 === -0 // true
```

Отличие заметно, например, при делении:

```
js
КопироватьРедактировать
1 / +0 // Infinity
1 / -0 // -Infinity
```

В памяти они отличаются **одним битом знака** (все остальные биты нули).

Сравнения и погрешности

- **Нельзя** полагаться на точное сравнение `a === b` для результатов вычислений с дробями. Используйте «эпсилон-сравнение», например (в JS):

```
js
КопироватьРедактировать
function nearlyEqual(a, b, eps = Number.EPSILON) {
  return Math.abs(a - b) <= eps * Math.max(1, Math.abs(a),
Math.abs(b));
}
```

- **NaN поведение**

```
js
КопироватьРедактировать
NaN === NaN          // false
Number.isNaN(NaN)    // true
```

- **Проверка на ± 0**

js

КопироватьРедактировать

```
0 === -0 // true
```

```
Object.is(0, -0) // false
```

```
1 / 0 // Infinity
```

```
1 / -0 // -Infinity
```

Приведение типов и смешанная арифметика

- В JS все арифметические операции над `Number` — в `double`. **BigInt** нельзя смешивать с `Number` без явного преобразования (ошибка).
- В C/Java при смешении `float` и `double` обычно происходит приведение `float` → `double`.

6. Строковые типы данных (особенно в JavaScript)

Что такое строка логически и физически

- **Строка** — последовательность символов (в абстракции). На практике есть разница между:
 - **кодировочная точка (Unicode code point)** — например U+1F60A;
 - **кодировочная единица (code unit)** — элемент внутреннего представления (в UTF-16 это 16-битные units);
 - **графемный кластер (grapheme cluster)** — «пользовательский символ» (например, буква + комбинирующий знак или сложный emoji).
- Ошибки часто связаны с тем, что стандартные API возвращают число кодировочных единиц, а не видимых символов.

JavaScript: внутреннее представление

- **JS хранит строки как последовательность UTF-16 code units.**
 - BMP (Basic Multilingual Plane, U+0000..U+FFFF) — один code unit.
 - Символы вне BMP (например, многие emoji, U+1F600+) представляются **суррогатной парой** — две 16-битные code units.

- `str.length` возвращает количество **кодowych единиц**, не количество «символов»/графем.
- **BMP** — это **подмножество символов Unicode** с номерами до U+FFFF.
- **UTF-16** — это **формат кодирования**, в котором символы из BMP занимают 1 блок, а символы вне BMP — 2 блока.

Unicode определяет BMP как все символы с кодами от U+0000 до U+FFFF.

js

КопироватьРедактировать

```
'a'.length // 1
'😊'.length // 2 (surrogate pair)
```

- Итерация `for...of` и `spread/Array.from` корректно разбивают строку по **unicode code points** (учитывая суррогатные пары), поэтому удобны для подсчёта «символов»:

js

КопироватьРедактировать

```
[...'😊'].length // 1
for (const ch of '😊') console.log(ch); // правильно выведет одну
«СИМВОЛ»
```

Основные операции в JS

- Конкатенация: `+`, шаблонные строки (template literals) — ``Hello ${name}``.
- Индексация: `str[i]` даёт **code unit** (не всегда полный символ).
- Извлечение подстроки: `slice`, `substring`, `substr` (последний deprecated).
- Поиск: `indexOf`, `includes`, `startsWith`, `endsWith`.
- Регулярные выражения: мощный инструмент для поиска/замены; для Unicode-aware паттернов используйте флаг `u` и Unicode property escapes `\p{...}` (ES2018+).
- Форматирование: `toUpperCase`, `toLowerCase`, `trim`, `padStart`, `padEnd`.
- Замена всех вхождений: `str.replaceAll('a', 'b')` (новее) или `str.replace(/a/g, 'b')`.
- Unicode-нормализация: `str.normalize('NFC' | 'NFD' | 'NFKC' | 'NFKD')`. Нужна для корректного сравнения строк с комбинирующими символами:

js

КопироватьРедактировать

```
'\u00E9' === 'e\u0301' // false
'\u00E9'.normalize('NFC') === 'e\u0301'.normalize('NFC') // true
```

Кодировки и преобразования

- **UTF-8** ↔ строка: в браузере/в среде есть `TextEncoder` и `TextDecoder`:

js

КопироватьРедактировать

```
const encoder = new TextEncoder();
const bytes = encoder.encode('Привет'); // Uint8Array (UTF-8)
const decoder = new TextDecoder();
console.log(decoder.decode(bytes));
```

- В Node.js есть `Buffer.from(str, 'utf8')` и `buffer.toString('utf8')`.

Мутируемость и производительность

- **Строки в JS неизменяемы (immutable)** — операции создают новые строки. Для частых мутаций используйте массивы или сборщик строк (в Java — `StringBuilder`). В JS частая стратегия: собрать кусочки в массив и `join('')` в конце; правда, современные движки оптимизируют конкатенацию +.
- Для больших текстов следите за памятью и обрезками.

Локализация, сортировка и сравнение

- Для корректной сортировки/сравнения по языку используйте `String.prototype.localeCompare` или `Intl.Collator`:

js

КопироватьРедактировать

```
['a', 'ä', 'z'].sort((a,b) => a.localeCompare(b, 'de')); // немецкий порядок
```

Безопасность и экранирование

- При вставке строк в HTML — обязательно экранировать/проверять (XSS).
- При работе с SQL — использовать параметризованные запросы (prepared statements), а не конкатенацию.

1. Что такое `charCodeAt()` в JavaScript

- `charCodeAt(index)` — метод строки, который возвращает **числовое значение UTF-16 code unit** (целое число от 0 до 65535) на позиции `index`.
- То есть он не возвращает "код символа" в привычном Unicode смысле, а именно 16-битное значение ячейки памяти, в которой хранится кусок строки.

Пример:

js

КопироватьРедактировать

```
let str = "A";
```

```
console.log(str.charCodeAt(0)); // 65 (U+0041)
```

Внутренне **JavaScript** хранит строки в **UTF-16**, а не в UTF-8.

7. Операции над данными строкового типа в языках программирования высокого уровня

Основные операции со строками в JavaScript и Python

Операция	JS пример	Python пример	Описание	
Конкатенация	<code>'hello' + ' ' + 'world'</code>	<code>'hello' + ' ' + 'world'</code>	Соединение строк	
Повторение строки	<code>'ha'.repeat(3)</code>	<code>'ha' * 3</code>	Повтор строки заданное число раз	
Длина строки	<code>'abc'.length</code>	<code>len('abc')</code>	Количество символов (в JS — code units)	
Индексация	<code>'abc'[1]</code>	<code>'abc'[1]</code>	Получить символ по индексу	
Извлечение подстроки	<code>'hello'.slice(1,4)</code>	<code>'hello'[1:4]</code>	Вырезать часть строки	
Поиск	<code>'hello'.indexOf('ll')</code>	<code>'hello'.find('ll')</code>	Найти позицию подстроки	
Проверка вхождения	<code>'hello'.includes('ll')</code>	<code>'ll' in 'hello'</code>	Есть ли подстрока	
Замена	<code>'hello'.replace('ll', 'y')</code>	<code>'hello'.replace('ll', 'y')</code>	Замена первого вхождения	
Замена всех	<code>'hello'.replaceAll('l', 'y')</code>	<code>re.sub('l', 'y', 'hello')</code>	Замена всех вхождений	
Разбиение строки	<code>'a,b,c'.split(',')</code>	<code>'a,b,c'.split(',')</code>	Разбить строку на массив	
Объединение массива в строку	<code>['a', 'b', 'c'].join('-')</code>	<code>'-'.join(['a', 'b', 'c'])</code>	Обратная операция	
Приведение регистра	<code>'hello'.toUpperCase(), 'hello'.toLowerCase()</code>	<code>'Hello'.upper(), 'hello'.lower()</code>	Приведение к верхнему/нижнему	
Удаление пробелов	<code>' hello '.trim()</code>	<code>' hello '.strip()</code>	Удаление пробелов по краям	
Шаблонные строки (интерполяция)	<code>`Name: \${name}`</code>	<code>f"Name: {name}"</code>	Вставка переменных в строки	

Особенности в JS и Python

- В JS строки **неизменяемы** (immutable), операции создают новые строки.
- В Python тоже immutable, для частых изменений используют `str.join` или `io.StringIO` для эффективности.
- В JS для Unicode символов вне BMP (👉) важно помнить, что индексирование по `str[i]` возвращает кодовую единицу, а не полноценный символ. Для корректной работы с символами используйте `for...of` или `Array.from()`.

8. Логические типы данных в языках программирования высокого уровня

Что такое логический тип (boolean)

- Логический тип — это тип данных с двумя значениями: `true` и `false`.
- Используется для выражения условий, флагов, переключателей.
- В большинстве языков именно так:
 - **JavaScript**: `true` и `false` (булевы литералы).
 - **Python**: `True` и `False`.
 - **Java, C#**: `boolean` тип с `true/false`.

Приведение к булеву типу

- В JS: в булевый тип приводятся все значения, например:
 - "Falsy" значения (которые приводятся к `false`): `0`, `""` (пустая строка), `null`, `undefined`, `NaN`, `false`.
 - Всё остальное — "truthy" (приводится к `true`).
- В Python: почти все объекты приводятся к `True`, кроме `False`, `None`, `0`, пустых коллекций (`' '`, `[]`, `{}`) и т.п.

Явное преобразование в Boolean

- Функция `Boolean(value)` — приводит значение к `true` или `false`.
- Оператор `!!value` — двойное отрицание, короткий способ привести к булеву значению.

js

КопироватьРедактировать

```
console.log(Boolean(0));    // false
console.log(Boolean("Hi")); // true
console.log(!!null);        // false
```

Проверка в условиях:

```
python
КопироватьРедактировать
if value:
    # если value истинно (truthy)
else:
    # если value ложно (falsy)
```

Явное преобразование к bool

- В Python есть функция `bool(value)`, которая приводит к `True` или `False`.

```
python
КопироватьРедактировать
print(bool(0))          # False
print(bool("Hello"))    # True
```

9. Операции над данными логического типа в языках программирования высокого уровня

Основные логические операции

Операция	JS символ	Python оператор	Описание
Логическое И	&&	and	Возвращает true, если оба операнда true
Логическое ИЛИ		or	Возвращает true, если хотя бы один операнд true

Логическое НЕ	!	not	Инвертирует значение
Побитовый AND	&	&	Побитовая операция (над числами)
Побитовый OR			Побитовая операция (над числами)
Побитовый XOR	^	^	Побитовая операция (над числами)

Пример в JavaScript

js

КопироватьРедактировать

```
let a = true;
let b = false;
```

```
console.log(a && b); // false
console.log(a || b); // true
console.log(!a);    // false
```

```
// Побитовые операции работают над целыми числами
console.log(5 & 3); // 1 (0101 & 0011 = 0001)
console.log(5 | 3); // 7 (0101 | 0011 = 0111)
console.log(5 ^ 3); // 6 (0101 ^ 0011 = 0110)
```

Особенности в JavaScript

- && и || — не просто возвращают true/false, а возвращают **одно из значений операндов** (короткое замыкание):

js

КопироватьРедактировать

```
console.log(false && 'Hello'); // false
console.log(true || 'Hello');  // true
```

- Это позволяет писать конструкции вроде:

js

КопироватьРедактировать

```
let name = inputName || "Guest";
```

- Побитовые операторы (&, |, ^) — работают над 32-битными целыми числами, поэтому перед операцией значения приводятся к 32-битным int.

Пример в Python

```
python
```

КопироватьРедактировать

```
a = True
```

```
b = False
```

```
print(a and b)  # False
```

```
print(a or b)   # True
```

```
print(not a)    # False
```

```
# Побитовые операции
```

```
print(5 & 3)     # 1
```

```
print(5 | 3)     # 7
```

```
print(5 ^ 3)     # 6
```

&& (логическое И):

- Вычисляет операнды слева направо.
- Если любой операнд является ложным (или приводится к false), оператор немедленно возвращает этот операнд, и вычисление остальных операндов прекращается (сокращенное вычисление).
- Если все операнды истинны (или приводятся к true), оператор возвращает значение последнего операнда.

|| (логическое ИЛИ):

- Вычисляет операнды слева направо.
- Если любой операнд является истинным (или приводится к true), оператор немедленно возвращает этот операнд, и вычисление остальных операндов прекращается (сокращенное вычисление).
- Если все операнды ложны (или приводятся к false), оператор возвращает значение последнего операнда.

10. Определение типов переменных в языках программирования высокого уровня

Важное понятие: динамическая типизация

- В языках высокого уровня (например, JavaScript и Python) переменные обычно **динамически типизированы** — то есть тип значения определяется во время выполнения программы, а не заранее.
- Переменная сама по себе **не имеет фиксированного типа**, а хранит ссылку на значение определённого типа.

JavaScript

- Объявление переменных:

js

КопироватьРедактировать

```
let a = 5;           // number
let b = "hello";     // string
let c = true;        // boolean
let d = null;        // object (специфика JS)
let e;               // undefined
```

- Тип можно узнать с помощью оператора `typeof`:

js

КопироватьРедактировать

```
console.log(typeof a); // "number"
console.log(typeof b); // "string"
console.log(typeof d); // "object"
```

- Тип может измениться при присваивании нового значения:

js

КопироватьРедактировать

```
let x = 10;      // number
x = "text";     // теперь string
```

Python

- Переменные объявляются просто присваиванием:

python

КопироватьРедактировать

```
a = 5           # int
b = "hello"     # str
c = True        # bool
```

- Тип можно узнать функцией `type()`:

python

КопироватьРедактировать

```
print(type(a)) # <class 'int'>
print(type(b)) # <class 'str'>
```

- Как и в JS, переменная может ссылаться на объекты разных типов в разное время.

Что такое статическая типизация?

- **Статическая типизация** — это когда тип переменной известен и фиксируется **на этапе компиляции** (до запуска программы).
- В таких языках компилятор проверяет типы, предотвращая ошибки с несовместимыми типами данных.
- После объявления тип переменной не может измениться (в большинстве языков).

Примеры языков со статической типизацией:

- **Java**
- **C#**
- **C++** (частично — есть динамические конструкции)
- **Rust**
- **Go**

- Swift
- Kotlin

Примеры популярных типов данных в разных языках

Тип данных	Java	C#	Python	JavaScript	Go
Целочисленные	<code>int</code> , <code>long</code> , <code>short</code> , <code>byte</code>	<code>int</code> , <code>long</code> , <code>byte</code>	<code>int</code> (динамический)	<code>number</code> (всё число)	<code>int</code> , <code>int64</code> , <code>uint8</code>
Вещественные	<code>float</code> , <code>double</code>	<code>float</code> , <code>double</code>	<code>float</code>	<code>number</code>	<code>float32</code> , <code>float64</code>
Логические	<code>boolean</code>	<code>bool</code>	<code>bool</code>	<code>boolean</code>	<code>bool</code>
Символы	<code>char</code>	<code>char</code>	<code>str</code> (один символ)	<code>string</code> (строка)	<code>rune</code>
Строки	<code>String</code>	<code>string</code>	<code>str</code>	<code>string</code>	<code>string</code>
Массивы и коллекции	массивы, <code>List</code>	массивы, <code>List</code>	<code>list</code> , <code>tuple</code> , <code>dict</code>	<code>Array</code> , <code>Object</code>	срезы (<code>slices</code>), массивы
Null / None	<code>null</code>	<code>null</code>	<code>None</code>	<code>null</code> / <code>undefined</code>	<code>nil</code>

Статическая vs Динамическая типизация

Характеристика	Статическая типизация	Динамическая типизация
Когда определяется тип	На этапе компиляции	Во время выполнения
Можно ли менять тип	Обычно нельзя	Можно (переменная может менять тип)
Проверка ошибок	На этапе компиляции	Во время выполнения
Примеры языков	Java, C#, Rust, Go	Python, JavaScript, Ruby

11. Области видимости переменных в языках программирования высокого уровня

Что такое область видимости?

Область видимости — это часть программы, где переменная доступна.

В JavaScript:

- **Глобальная область видимости** — переменные, объявленные вне функций или блоков, доступны везде.

- **Функциональная область видимости** — переменные, объявленные с `var` внутри функции, видимы только внутри функции.
- **Блочная область видимости** — переменные с `let` и `const` видимы только внутри блока `{ ... }`, например, внутри цикла, условия.

Пример:

```
js
КопироватьРедактировать
let x = 1;           // глобальная
function f() {
  var y = 2;         // локальная функция
  if (true) {
    let z = 3;       // блочная область
    console.log(z); // 3
  }
  // console.log(z); // Ошибка: z не видна здесь
}
f();
// console.log(y);    // Ошибка: y не видна здесь
```

В Python:

- Есть глобальная и локальная области видимости (функциональная).
- Внутри функции переменные — локальные, если не объявлены как `global` или `nonlocal`.

Пример:

```
python
КопироватьРедактировать
x = 1 # глобальная

def f():
    y = 2 # локальная
    print(y)

f()
print(x) # 1
# print(y) # Ошибка: y не определена вне функции
```

- Для доступа к глобальной переменной из функции используется `global`:

python

КопироватьРедактировать

```
x = 1
```

```
def f():
```

```
    global x
```

```
    x = 5
```

```
f()
```

```
print(x)  # 5
```

12. Константы в языках программирования высокого уровня

В JavaScript:

- Введены в ES6 ключевым словом `const`.
- Константа – это переменная, которая **не может быть переназначена**.

js

КопироватьРедактировать

```
const PI = 3.14;
```

```
PI = 3.1415; // Ошибка: нельзя переназначать const
```

- Однако, если `const` содержит объект или массив, изменять свойства или элементы можно:

js

КопироватьРедактировать

```
const obj = {a: 1};
```

```
obj.a = 2; // Это допустимо
```

В Python:

- Явных констант нет – все переменные можно переназначать.
- По соглашению, **константы записывают большими буквами**:

python

[Копировать](#)[Редактировать](#)

```
PI = 3.14
```

```
PI = 3.1415 # Можно, но не рекомендуется
```

Java

- Константы объявляются с помощью ключевого слова `final` и обычно – `static final` в классах:

java

[Копировать](#)[Редактировать](#)

```
public static final double PI = 3.14159;
```

- После объявления `final` переменную нельзя переназначить.

C#

- Ключевое слово `const` для констант:

csharp

[Копировать](#)[Редактировать](#)

```
const double PI = 3.14159;
```

- Также есть `readonly` – значение можно установить в конструкторе, но не менять после.

Python

- Нет встроенной поддержки констант.
- По соглашению – именуют с заглавными буквами (например, `PI = 3.14`), но это не запрещает менять.

- Для имитации констант можно использовать:
 - Модули с переменными, которые не изменяют.
 - Использовать `@property` в классах без сеттера.
 - Библиотеки, например `const` (сторонние).

C/C++

- Ключевое слово `const`:

c

КопироватьРедактировать

```
const int MAX = 100;
```

- `const` запрещает изменение значения после инициализации.

13. Оператор присваивания в языках программирования высокого уровня

JavaScript

- Основной оператор присваивания — `=`

js

КопироватьРедактировать

```
let a = 5;
```

```
let b = a; // b теперь равно 5
```

- Есть также **сокращённые операторы присваивания с арифметическими операциями**:

js

КопироватьРедактировать

```
a += 3; // эквивалентно a = a + 3
```

```
a -= 2; // a = a - 2
```

```
a *= 4; // a = a * 4
a /= 2; // a = a / 2
a %= 3; // a = a % 3
```

- Эти операторы работают с числами, строками (конкатенация с +=), и другими типами, где это определено.

Python

- Аналогично, основной оператор присваивания — =

```
python
КопироватьРедактировать
a = 5
b = a
```

- Есть сокращённые формы, аналогичные JavaScript:

```
python
КопироватьРедактировать
a += 3 # a = a + 3
a -= 2
a *= 4
a /= 2
a %= 3
```

Языки со статической типизацией (Java, C#, C++)

- Также используют = для присваивания.
- Сокращённые операторы (например, +=, -=, и т.д.) поддерживаются.
- Пример на Java:

```
java
КопироватьРедактировать
int a = 10;
a += 5; // теперь a = 15
```

- Важно: в статически типизированных языках переменная имеет фиксированный тип, присваиваемое значение должно соответствовать типу.

14. Условный оператор в языках программирования высокого уровня

JavaScript

- Основной условный оператор — `if`

js

КопироватьРедактировать

```
if (condition) {  
    // блок кода, если условие true  
} else if (otherCondition) {  
    // если первое условие false, но это true  
} else {  
    // если все условия false  
}
```

- Условие приводится к булеву типу (truthy/falsy).
- Пример:

js

КопироватьРедактировать

```
let x = 10;  
if (x > 5) {  
    console.log("Больше 5");  
} else {  
    console.log("Меньше или равно 5");  
}
```

Python

- Аналогично, синтаксис:

```
python
КопироватьРедактировать
if condition:
    # блок кода
elif other_condition:
    # другой блок
else:
    # иначе
```

- Отступы определяют блоки кода.

```
python
КопироватьРедактировать
x = 10
if x > 5:
    print("Больше 5")
else:
    print("Меньше или равно 5")
```

Языки со статической типизацией

- Java, C#, C++:

```
java
КопироватьРедактировать
if (condition) {
    // тело
} else if (otherCondition) {
    // тело
} else {
    // тело
}
```

- Условие должно быть логическим выражением (например, boolean в Java).

15. Сокращённые формы условного оператора в языках программирования высокого уровня

Тернарный оператор (условный оператор в выражении)

- Позволяет записать условие кратко в виде выражения.

JavaScript

```
js
КопироватьРедактировать
let result = (condition) ? valueIfTrue : valueIfFalse;
```

Пример:

```
js
КопироватьРедактировать
let age = 18;
let canVote = (age >= 18) ? "Yes" : "No";
console.log(canVote); // "Yes"
```

Python

- В Python тернарный оператор выглядит так:

```
python
КопироватьРедактировать
result = valueIfTrue if condition else valueIfFalse
```

Пример:

```
python
```

КопироватьРедактировать

```
age = 18
can_vote = "Yes" if age >= 18 else "No"
print(can_vote) # "Yes"
```

Другие сокращения в JavaScript

- Логические операторы с коротким замыканием:

js

КопироватьРедактировать

```
// Выполнить функцию, если условие истинно
condition && doSomething();
```

```
// Присвоить значение по умолчанию, если переменная falsy
let name = inputName || "Guest";
```

В языках со статической типизацией

- Тернарный оператор похож на JS:

java

КопироватьРедактировать

```
String result = (condition) ? "Yes" : "No";
```

Итог

Оператор	JavaScript	Python	Java/C#/C++
Условный оператор	if...else if...else	if...elif...else	if...else if...else
Тернарный оператор	(cond) ? valTrue : valFalse	valTrue if cond else valFalse	(cond) ? valTrue : valFalse
Присваивание	=	=	=
Сокращённые операторы	+=, -=, *=, /= и др.	Аналогично	Аналогично

16. Цикл с предусловием (while loop)

Что такое цикл с предусловием?

- Цикл, который сначала проверяет условие, и **если оно истинно** — **выполняет тело цикла**.
- Если условие изначально ложно — тело цикла не выполнится ни разу.

JavaScript

```
js
КопироватьРедактировать
while (condition) {
  // тело цикла
}
```

Пример:

```
js
КопироватьРедактировать
let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

Python

```
python
КопироватьРедактировать
while condition:
  # тело цикла
```

Пример:

python

КопироватьРедактировать

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Языки со статической типизацией (Java, C#, C++)

java

КопироватьРедактировать

```
while (condition) {
    // тело цикла
}
```

Особенность — `condition` должно быть булевым выражением.

17. Цикл с постусловием (do...while loop)

Что такое цикл с постусловием?

- Тело цикла выполняется **сначала один раз**, а потом проверяется условие. Если условие истинно — повторяется.
- Гарантированно выполняется минимум один раз.

JavaScript

js

КопироватьРедактировать

```
do {
    // тело цикла
```



```
} while (condition);
```

Пример:

```
js
КопироватьРедактировать
let i = 0;
do {
  console.log(i);
  i++;
} while (i < 5);
```

Python

- В Python **нет встроенного оператора do...while**.
- Эту логику имитируют, например, так:

```
python
КопироватьРедактировать
i = 0
while True:
    print(i)
    i += 1
    if not (i < 5):
        break
```

Языки со статической типизацией

- Java, C#, C++ поддерживают do...while:

```
java
КопироватьРедактировать
do {
    // тело
} while (condition);
```

18. Цикл с параметром (for loop)

Что такое цикл с параметром?

- Цикл, который повторяет тело определённое количество раз или по коллекции, с управлением переменной-счётчиком.

JavaScript

- Традиционный цикл for:

```
js
КопироватьРедактировать
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

- Цикл по элементам массива (ES6+):

```
js
КопироватьРедактировать
for (let item of array) {
  console.log(item);
}
```

Python

- Цикл for в Python итеративный — перебирает элементы из итерируемого объекта:

```
python
КопироватьРедактировать
for i in range(5):
  print(i)
```

- Перебор элементов списка:

python

[Копировать](#)[Редактировать](#)

```
for item in my_list:  
    print(item)
```

Языки со статической типизацией

- Java:

java

[Копировать](#)[Редактировать](#)

```
for (int i = 0; i < 5; i++) {  
    System.out.println(i);  
}
```

- Java (for-each цикл):

java

[Копировать](#)[Редактировать](#)

```
for (String item : array) {  
    System.out.println(item);  
}
```

- C#:

csharp

[Копировать](#)[Редактировать](#)

```
for (int i = 0; i < 5; i++) {  
    Console.WriteLine(i);  
}
```

// foreach

```
foreach (var item in collection) {  
    Console.WriteLine(item);  
}
```

- C++:

```

cpp
КопироватьРедактировать
for (int i = 0; i < 5; i++) {
    std::cout << i << std::endl;
}

// range-based for (C++11+)
for (auto& item : collection) {
    std::cout << item << std::endl;
}

```

Итог

Тип цикла	JavaScript	Python	Статически типизированные языки
Цикл с предусловием	<code>while (condition) {}</code>	<code>while condition:</code>	<code>while (condition) {}</code>
Цикл с постусловием	<code>do { } while(condition);</code>	Нет встроенного оператора, имитация через <code>while True + break</code>	<code>do { } while(condition);</code>
Цикл с параметром	<code>for(init; cond; inc){}</code> и <code>for...of</code>	<code>for var in iterable:</code>	<code>for(init; cond; inc){}</code> и <code>foreach</code>

методы вроде `forEach`, `map` и подобные **технически не являются классическими циклами**, но по сути выполняют итерацию — то есть обход элементов коллекции с вызовом функции.

3. Метод `forEach` (JavaScript)

- Метод массива, принимает функцию и вызывает её для каждого элемента.

```

js
КопироватьРедактировать

```

```
const arr = [10, 20, 30];
arr.forEach((item, index) => {
  console.log(item, index);
});
```

- Аргументы функции: значение элемента, индекс, сам массив.
- Это **не классический цикл**, а метод перебора с callback, но по смыслу выполняет итерацию.

4. Метод map (JavaScript)

- Метод для **создания нового массива**, применяя функцию к каждому элементу исходного.

```
js
КопироватьРедактировать
const arr = [1, 2, 3];
const doubled = arr.map(x => x * 2);
console.log(doubled); // [2, 4, 6]
```

•

range(start, stop, step)

19. Оператор множественного выбора

Что это?

- Оператор множественного выбора — конструкция, позволяющая выбрать один из нескольких вариантов исполнения кода в зависимости от значения выражения.

JavaScript

- Основной — оператор `switch`.

js

КопироватьРедактировать

```
let day = 3;

switch (day) {
  case 1:
    console.log("Понедельник");
    break;
  case 2:
    console.log("Вторник");
    break;
  case 3:
    console.log("Среда");
    break;
  default:
    console.log("Неизвестный день");
}
```

- `case` — варианты, `default` — вариант по умолчанию.
- Важно использовать `break`, чтобы выйти из `switch`.

Python

- В Python до версии 3.10 отсутствует встроенный оператор `switch`.
- Обычно для множественного выбора используют словари с функциями или `if-elif-else`:

python

КопироватьРедактировать

```
def monday():
    print("Понедельник")

def tuesday():
    print("Вторник")

def wednesday():
```

```
print("Среда")

switch = {
    1: monday,
    2: tuesday,
    3: wednesday
}

day = 3
switch.get(day, lambda: print("Неизвестный день"))()
```

- Начиная с Python 3.10 появился оператор `match` (структурное сопоставление):

```
python
КопироватьРедактировать
day = 3

match day:
    case 1:
        print("Понедельник")
    case 2:
        print("Вторник")
    case 3:
        print("Среда")
    case _:
        print("Неизвестный день")
```

Языки со статической типизацией

- Java, C#, C++ поддерживают `switch` с синтаксисом, похожим на JS, но с некоторыми отличиями.

20. Тип данных «Структура»

Что такое структура?

- Структура — пользовательский тип данных, объединяющий несколько полей (различных типов) под одним именем.

JavaScript

- Нет встроенного типа `struct`, но есть объекты, которые выполняют схожую роль:

js

КопироватьРедактировать

```
const person = {  
  name: "Alice",  
  age: 30,  
  isStudent: false  
};
```

Python

- В Python аналог — словари, классы и `namedtuple`:

python

КопироватьРедактировать

Словарь

```
person = {"name": "Alice", "age": 30, "is_student": False}
```

Namedtuple (из модуля `collections`)

```
from collections import namedtuple
```

```
Person = namedtuple('Person', ['name', 'age', 'is_student'])
```

```
p = Person("Alice", 30, False)
```

- Также можно использовать класс:

python

КопироватьРедактировать

```
class Person:
```

```
    def __init__(self, name, age, is_student):
```



```
self.name = name
self.age = age
self.is_student = is_student
```

```
p = Person("Alice", 30, False)
```

Языки со статической типизацией

- В C, C++ есть ключевое слово `struct`:

с
КопироватьРедактировать

```
struct Person {
    char name[50];
    int age;
    bool is_student;
};
```

- В Java нет `struct`, но есть классы (классы выполняют эту роль).

Как работать с `struct` в C/C++ (на примере `Person`)

Объявление структуры

с
КопироватьРедактировать

```
struct Person {
    char name[50];
    int age;
    bool is_student;
};
```

Создание переменной структуры

с

КопироватьРедактировать

```
struct Person p1;
```

Или с помощью typedef для удобства (C):

с

КопироватьРедактировать

```
typedef struct {  
    char name[50];  
    int age;  
    bool is_student;  
} Person;  
  
Person p1;
```

Заполнение полей

с

КопироватьРедактировать

```
#include <stdio.h>  
#include <string.h>  
#include <stdbool.h>  
  
int main() {  
    struct Person p1;  
  
    // Заполнение имени – функция strcpy копирует строку  
    strcpy(p1.name, "Alice");  
  
    p1.age = 30;  
    p1.is_student = false;  
  
    printf("Name: %s\nAge: %d\nStudent: %d\n", p1.name, p1.age,  
p1.is_student);  
  
    return 0;  
}
```

Обращение к полям

- Используйте оператор `.` (точка):

```
с
КопироватьРедактировать
p1.age = 25;
printf("%s is %d years old\n", p1.name, p1.age);
```

Полный пример:

```
с
КопироватьРедактировать
#include <stdio.h>
#include <string.h>
#include <stdbool.h> // для bool в C

struct Person {
    char name[50];
    int age;
    bool is_student;
};

int main() {
    struct Person p1;

    strcpy(p1.name, "Alice");
    p1.age = 25;
    p1.is_student = true;

    printf("Name: %s\nAge: %d\nStudent: %d\n", p1.name, p1.age,
p1.is_student);

    return 0;
}
```

- Само слово `Person` — это тип, **к нему напрямую обратиться нельзя** (нельзя написать просто `Person = ...`).
- С помощью типа `Person` создают **переменные** (объекты), например `p1`, `p2`, `student`.
- С этими переменными можно работать, обращаться к полям, изменять значения.

21. Подпрограммы в языках программирования высокого уровня

Что такое подпрограммы?

- Это именованные блоки кода (функции, процедуры), которые можно вызывать из разных частей программы.

JavaScript

- Функции:

```
js
КопироватьРедактировать
function greet(name) {
  return "Hello, " + name;
}

console.log(greet("Alice"));
```

- Также есть стрелочные функции:

```
js
КопироватьРедактировать
const greet = (name) => "Hello, " + name;
```

Python

- Определение функции с помощью def:

python

КопироватьРедактировать

```
def greet(name):  
    return "Hello, " + name
```

```
print(greet("Alice"))
```

Языки со статической типизацией

- Java:

java

КопироватьРедактировать

```
public String greet(String name) {  
    return "Hello, " + name;  
}
```

- C#:

csharp

КопироватьРедактировать

```
string Greet(string name) {  
    return "Hello, " + name;  
}
```

- C++:

cpp

КопироватьРедактировать

```
std::string greet(std::string name) {  
    return "Hello, " + name;  
}
```

Подпрограммами могут быть еще методы класса, которые вызываются у объектов

22. Массивы в языках программирования высокого уровня

Что такое массив?

- Массив – это упорядоченная коллекция элементов одного или разных типов, к которым можно обращаться по индексу.

Python

- В Python массивом обычно считается список (`list`) – динамическая структура, хранящая элементы в порядке добавления.

python

[Копировать](#)[Редактировать](#)

```
arr = [10, 20, 30, 40]
```

- Элементы могут иметь разные типы, но чаще используют однородные.
- Индексация с 0: `arr[0]` – первый элемент.

JavaScript

- В JS массив – объект `Array` с динамическим размером.

js

[Копировать](#)[Редактировать](#)

```
let arr = [10, 20, 30, 40];
```

- Также динамический, элементы могут быть разных типов.
- Индексация с 0: `arr[0]`.

Языки со строгой типизацией (Java, C++, C#)

- Массивы имеют фиксированный размер и элементы одного типа.

java

КопироватьРедактировать

```
int[] arr = new int[4];  
arr[0] = 10;
```

- Размер массива задаётся при создании и не меняется (в отличие от динамических списков Python или JS).

JavaScript предоставляет очень богатый набор методов и операторов для работы с массивами. Вот основные из них:

Создание массива

js

КопироватьРедактировать

```
let arr = [1, 2, 3];  
let arr2 = new Array(4, 5, 6);
```

Добавление и удаление элементов

- **push** — добавить элемент в конец массива

js

КопироватьРедактировать

```
arr.push(4); // arr = [1,2,3,4]
```

- **pop** — удалить последний элемент

js

КопироватьРедактировать

```
arr.pop(); // arr = [1,2,3]
```

- **unshift** – добавить элемент в начало массива

```
js
КопироватьРедактировать
arr.unshift(0); // arr = [0,1,2,3]
```

- **shift** – удалить первый элемент

```
js
КопироватьРедактировать
arr.shift(); // arr = [1,2,3]
```

Доступ по индексу и замена элементов

```
js
КопироватьРедактировать
let first = arr[0]; // доступ
arr[1] = 10;        // замена
```

Поиск элементов

- **indexOf** – индекс первого вхождения

```
js
КопироватьРедактировать
arr.indexOf(10); // если нет – -1
```

- **includes** – проверка наличия элемента

```
js
КопироватьРедактировать
arr.includes(10); // true/false
```

- **find** – найти первый элемент, удовлетворяющий условию

```
js
```


КопироватьРедактировать

```
arr.find(x => x > 5);
```

- **findIndex** – индекс первого, удовлетворяющего условию

js

КопироватьРедактировать

```
arr.findIndex(x => x > 5);
```

Перебор массива

- **forEach**

js

КопироватьРедактировать

```
arr.forEach((el, i) => console.log(i, el));
```

- **map** – возвращает новый массив, применяя функцию к каждому элементу

js

КопироватьРедактировать

```
let doubled = arr.map(x => x * 2);
```

- **filter** – возвращает новый массив с элементами, прошедшими фильтр

js

КопироватьРедактировать

```
let evens = arr.filter(x => x % 2 === 0);
```

- **reduce** – свёртка массива к одному значению

js

КопироватьРедактировать

```
let sum = arr.reduce((acc, x) => acc + x, 0);
```

Сортировка

- **sort** – сортирует массив (по умолчанию как строки)

js

КопироватьРедактировать

```
arr.sort(); // сортировка по алфавиту
```

// числовая сортировка

```
arr.sort((a, b) => a - b);
```

Изменение структуры массива

- **splice** – удаляет/добавляет элементы в любом месте массива

js

КопироватьРедактировать

```
arr.splice(1, 2, 100, 200); // удалить 2 элемента с индекса 1,  
вставить 100,200
```

- **slice** – возвращает копию части массива (не изменяет исходный)

js

КопироватьРедактировать

```
let part = arr.slice(1, 3); // с 1 по 2 индекс (3 не включается)
```

Объединение и преобразование

- **concat** – объединить массивы

js

КопироватьРедактировать

```
let newArr = arr.concat([5,6]);
```

- **join** – преобразовать в строку

js

КопироватьРедактировать

```
let str = arr.join(", ");
```

- **flat** – "выравнивание" вложенных массивов (ES2019+)

js

КопироватьРедактировать

```
let nested = [1, [2, 3], [4, [5]]];  
nested.flat(2); // [1, 2, 3, 4, 5]
```

Другие полезные методы

- **reverse** – перевернуть массив
- **every** – проверяет, что все элементы удовлетворяют условию
- **some** – проверяет, что хотя бы один элемент удовлетворяет условию

23. Алгоритм поиска максимального элемента в массиве

Словесное объяснение

- Идём по массиву с начала до конца.
- Сохраняем текущее максимальное значение (например, первый элемент).
- Сравниваем каждый следующий элемент с текущим максимумом.
- Если нашли больше – обновляем максимум.
- После прохода по всем элементам максимум – это искомое значение.

Временная и пространственная сложность

Алгоритм	Время	Память
Линейный поиск	$O(n)$	$O(1)$

- **Время:** проходим по массиву один раз – линейная сложность.
- **Память:** используем всего одну дополнительную переменную – константная.

Код на JavaScript (линейный поиск максимума)

```
js
КопироватьРедактировать
function findMax(arr) {
  if (arr.length === 0) return null; // пустой массив

  let max = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
      max = arr[i];
    }
  }
  return max;
}
```

Альтернативный алгоритм – рекурсивный поиск (пример, но менее эффективен)

```
js
КопироватьРедактировать
function findMaxRecursive(arr, index = 0) {
  if (index === arr.length - 1) return arr[index];
  const maxRest = findMaxRecursive(arr, index + 1);
  return arr[index] > maxRest ? arr[index] : maxRest;
}
```

- Время – $O(n)$, но из-за рекурсии потребляет память на стек ($O(n)$).

24. Алгоритм поиска минимального элемента в массиве

Словесное объяснение

- Аналогично поиску максимума: идём по массиву, храним текущий минимум, обновляем при встрече меньшего элемента.

Временная и пространственная сложность

Алгоритм	Время	Память
Линейный поиск	$O(n)$	$O(1)$

Код на JavaScript (линейный поиск минимума)

```
js
КопироватьРедактировать
function findMin(arr) {
  if (arr.length === 0) return null;

  let min = arr[0];
  for (let i = 1; i < arr.length; i++) {
    if (arr[i] < min) {
      min = arr[i];
    }
  }
  return min;
}
```

Кратко о других способах (без кода)

Использование встроенных функций

- В JavaScript:

js

КопироватьРедактировать

```
Math.max(...arr);
```

```
Math.min(...arr);
```

- В Python:

python

КопироватьРедактировать

```
max(arr)
```

```
min(arr)
```

Встроенные функции, как правило, имеют линейную сложность и оптимизированы.

Сложность и память

- Поиск `max` или `min` всегда требует просмотра элементов хотя бы один раз ($O(n)$) – нельзя сделать быстрее без дополнительной структуры.
- Память для поиска константна ($O(1)$) – храним только текущий максимум/минимум.

23–24) Другие способы поиска максимума и минимума в массиве (кроме простого линейного)

1. Использование встроенных функций

JavaScript

- `Math.max` и `Math.min` с оператором `spread`:

js

КопироватьРедактировать

```
let max = Math.max(...arr);  
let min = Math.min(...arr);
```

- Временная сложность: $O(n)$, так как функция должна пройти весь массив.

2. Сортировка и выбор первого/последнего элемента

- Отсортировать массив и взять первый (минимум) или последний (максимум) элемент.

js

КопироватьРедактировать

```
let sortedAsc = [...arr].sort((a,b) => a - b);  
let min = sortedAsc[0];  
let max = sortedAsc[sortedAsc.length - 1];
```

- Временная сложность: $O(n \log n)$ из-за сортировки.
- Пространственная сложность: $O(n)$ — создаём копию массива.

3. Поиск максимума/минимума с помощью reduce

js

КопироватьРедактировать

```
let max = arr.reduce((acc, val) => val > acc ? val : acc, arr[0]);  
let min = arr.reduce((acc, val) => val < acc ? val : acc, arr[0]);
```

- Временная сложность: $O(n)$.
- Используется функциональный стиль.

4. Алгоритм "разделяй и властвуй"

- Можно разбить массив на части, найти максимум в каждой, а потом из них общий максимум.
- Полезно для распараллеливания.
- Для последовательного кода — сложнее, обычно уступает простому линейному.

5. Для отсортированных массивов

- Если массив уже отсортирован по возрастанию, минимум — первый элемент, максимум — последний, поиск $O(1)$.

25. Алгоритмы сортировки массива

1) Сортировка пузырьком (Bubble Sort)

Идея:

Повторно проходить по массиву, сравнивая соседние элементы и меняя их местами, если они в неправильном порядке. Каждый проход «всплывает» максимум в конец.

Временная сложность: $O(n^2)$ — плохая, т.к. два вложенных цикла.

Память: $O(1)$ — сортировка на месте.

JS-код:

js

КопироватьРедактировать

```
function bubbleSort(arr) {
  let n = arr.length;
  for (let i = 0; i < n - 1; i++) {
    for (let j = 0; j < n - 1 - i; j++) {
      if (arr[j] > arr[j + 1]) {
        // поменять местами
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
      }
    }
  }
  return arr;
}
```

2) Сортировка вставками (Insertion Sort)

Идея:

Берём элементы по одному и вставляем их в уже отсортированную часть слева.

Время: $O(n^2)$ в худшем, $O(n)$ если массив почти отсортирован.

Память: $O(1)$.

JS-код:

js

КопироватьРедактировать

```
function insertionSort(arr) {
  for (let i = 1; i < arr.length; i++) {
    let key = arr[i];
    let j = i - 1;
```

```

    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j--;
    }
    arr[j + 1] = key;
}
return arr;
}

```

3) Быстрая сортировка (Quick Sort)

Идея:

Выбирается опорный элемент (pivot), массив делится на части меньше и больше pivot, рекурсивно сортируем части.

Время: В среднем $O(n \log n)$, худший — $O(n^2)$ (редко).

Память: $O(\log n)$ из-за рекурсии.

JS-код:

js

КопироватьРедактировать

```

function quickSort(arr) {
    if (arr.length <= 1) return arr;

    const pivot = arr[arr.length - 1];
    const left = [];
    const right = [];

    for (let i = 0; i < arr.length - 1; i++) {
        if (arr[i] < pivot) left.push(arr[i]);
        else right.push(arr[i]);
    }

    return [...quickSort(left), pivot, ...quickSort(right)];
}

```

4) Сортировка слиянием (Merge Sort)

Идея:

Разделить массив пополам, рекурсивно отсортировать части, затем слить их обратно в отсортированный массив.

Время: Всегда $O(n \log n)$.

Память: $O(n)$ — используется дополнительный массив для слияния.

JS-код:

js

КопироватьРедактировать

```
function mergeSort(arr) {
  if (arr.length <= 1) return arr;

  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));

  return merge(left, right);
}

function merge(left, right) {
  const result = [];
  let i = 0, j = 0;
  while (i < left.length && j < right.length) {
    if (left[i] < right[j]) {
      result.push(left[i++]);
    } else {
      result.push(right[j++]);
    }
  }
  return result.concat(left.slice(i)).concat(right.slice(j));
}
```

26. Алгоритм двоичного поиска в отсортированном массиве

Идея:

- Массив отсортирован.
- Сравниваем искомое значение с элементом посередине.
- Если искомое меньше — ищем в левой половине, иначе — в правой.
- Повторяем, уменьшая область поиска вдвое, пока не найдём или не убедимся в отсутствии элемента.

Временная сложность: $O(\log n)$ — очень эффективно для больших массивов.

JS-код:

```
js
КопироватьРедактировать
function binarySearch(arr, target) {
  let left = 0;
  let right = arr.length - 1;

  while (left <= right) {
    let mid = Math.floor((left + right) / 2);

    if (arr[mid] === target) {
      return mid; // индекс найденного элемента
    } else if (arr[mid] < target) {
      left = mid + 1;
    } else {
      right = mid - 1;
    }
  }
}
```

```
    return -1; // элемент не найден
}
```

27. Что такое структура данных «дек»?

Определение

Дек (Deque, double-ended queue) — это структура данных, которая позволяет добавлять и удалять элементы с **обоих концов**: и с головы (начала), и с хвоста (конца).

Основные операции

Операция	Описание
push_front / unshift	Добавить элемент в начало
push_back / push	Добавить элемент в конец
pop_front / shift	Удалить элемент из начала
pop_back / pop	Удалить элемент из конца
peek_front	Посмотреть элемент в начале (не удаляя)
peek_back	Посмотреть элемент в конце (не удаляя)
Проверка пустоты	Проверить, пуст ли дек

Основные операции с деками (Deque)

Операция	Java (<code>ArrayDeque</code>)	C++ (<code>std::deque</code>)	Комментарии
Добавление в начало (<code>push_front</code>)	$O(1)$ (амортизированное)	$O(1)$ (амортизированное)	Быстро, без сдвигов
Добавление в конец (<code>push_back</code>)	$O(1)$ (амортизированное)	$O(1)$ (амортизированное)	
Удаление из начала (<code>pop_front</code>)	$O(1)$	$O(1)$	
Удаление из конца (<code>pop_back</code>)	$O(1)$	$O(1)$	
Доступ по индексу (<code>operator[]</code>)	$O(1)$	$O(1)$	Для <code>std::deque</code> доступ за константное время
Проверка пустоты	$O(1)$	$O(1)$	

JavaScript

- В JS нет встроенного типа «дек», но можно использовать массив (`Array`), так как у него есть методы для работы с концами:

js

КопироватьРедактировать

```
let deque = [];
```

```
// Добавление
```

```
deque.push(1);           // в конец
```

```
deque.unshift(0);       // в начало
```

```
// Удаление
```

```
let fromEnd = deque.pop(); // из конца
```

```
let fromStart = deque.shift(); // из начала
```

- Но операции `unshift` и `shift` имеют **линейную** сложность $O(n)$, т.к. сдвигают элементы. Для эффективного дека обычно используют связные списки или специальные структуры.

Python

- В Python есть встроенный модуль `collections` с типом `deque`, который реализован эффективно и поддерживает все операции за $O(1)$:

python

КопироватьРедактировать

```
from collections import deque
```

```
dq = deque()
```

```
dq.append(1)          # добавить в конец
```

```
dq.appendleft(0)      # добавить в начало
```

```
dq.pop()              # удалить из конца
```

```
dq.popleft()          # удалить из начала
```

Языки со строгой типизацией (Java, C++)

- В Java есть класс `ArrayDeque`:

java

КопироватьРедактировать

```
import java.util.ArrayDeque;
```

```
ArrayDeque<Integer> dq = new ArrayDeque<>();
```

```
dq.addFirst(0); // добавить в начало
```

```
dq.addLast(1);  // добавить в конец
```

```
dq.removeFirst();
```

```
dq.removeLast();
```

- В C++ есть `std::deque` из STL:

cpp

КопироватьРедактировать

```
#include <deque>
```

```
std::deque<int> dq;
```

```
dq.push_back(1);  
dq.push_front(0);
```

```
dq.pop_back();  
dq.pop_front();
```

28. Что такое структура данных «приоритетная очередь»?

Определение

Приоритетная очередь (Priority Queue) — структура данных, которая хранит элементы с приоритетом и позволяет:

- Быстро добавлять элементы.
- Извлекать элемент с **наивысшим приоритетом** (обычно минимальное или максимальное значение).

Основные операции

Операция	Описание
insert / enqueue	Добавить элемент с приоритетом
extract_max / extract_min	Извлечь элемент с наивысшим приоритетом
peek	Посмотреть элемент с наивысшим приоритетом
Проверка пустоты	Проверить, пуста ли очередь

Основные операции

Операция	Python (<code>heapq</code>)	Java (<code>PriorityQueue</code>)	C++ (<code>std::priority_queue</code>)	Комментарии
Вставка (enqueue)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Поддержание кучи
Извлечение минимального (pop) или максимального	$O(\log n)$	$O(\log n)$	$O(\log n)$	В Python <code>heapq</code> — минимальный элемент
Просмотр максимума/минимума (peek)	$O(1)$	$O(1)$	$O(1)$	Корень кучи
Проверка пустоты	$O(1)$	$O(1)$	$O(1)$	

Реализация

Чаще всего приоритетная очередь реализуется с помощью **кучи (heap)** — двоичной кучи, которая поддерживает структуру, где корень — максимум или минимум.

JavaScript

- В JS нет встроенного приоритетного очереди, но можно реализовать:

```
js
КопироватьРедактировать
class PriorityQueue {
  constructor() {
    this.heap = [];
  }

  // Добавление элемента и "всплытие" его в куче
  enqueue(value) {
    this.heap.push(value);
    this._bubbleUp();
  }

  // Извлечение максимума и перестройка кучи
  dequeue() {
    const max = this.heap[0];
    const end = this.heap.pop();
    if (this.heap.length > 0) {
      this.heap[0] = end;
      this._sinkDown();
    }
  }
}
```

```

    }
    return max;
}

_bubbleUp() { /* ... */ }
_sinkDown() { /* ... */ }
}

```

(Для полноты надо реализовать `_bubbleUp` и `_sinkDown` — операции поддержания свойства кучи.)

Python

- В Python есть встроенный модуль `heapq` для мин-кучи (минимальный приоритет):

```

python
КопироватьРедактировать
import heapq

pq = []
heapq.heappush(pq, 10)
heapq.heappush(pq, 5)
heapq.heappush(pq, 20)

min_elem = heapq.heappop(pq)  # 5

```

- Для максимальной очереди можно хранить отрицательные значения.

Языки со строгой типизацией

- Java: `PriorityQueue` из `java.util`

```

java
КопироватьРедактировать
import java.util.PriorityQueue;

```

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(10);
pq.add(5);

int min = pq.poll(); // извлечь минимальный элемент
```

- C++: `std::priority_queue` (максимальная очередь по умолчанию)

```
cpp
КопироватьРедактировать
#include <queue>
#include <vector>

std::priority_queue<int> pq;
pq.push(10);
pq.push(5);

int max = pq.top();
pq.pop();
```

1. Можно ли в приоритетной очереди извлечь и минимум, и максимум?

Стандартная приоритетная очередь устроена так, что:

- Она всегда быстро возвращает **либо минимум, либо максимум** (в зависимости от реализации).
- Обычно поддерживается только **один "крайний" элемент** — либо минимальный, либо максимальный.

Почему?

Потому что приоритетная очередь реализуется через кучу (heap), где корень — либо минимальный (min-heap), либо максимальный (max-heap) элемент.

- Извлечь **минимум** быстро можно из мин-кучи.

- Извлечь **максимум** быстро можно из макс-кучи.

Что если нужна и минимальная, и максимальная извлекать?

Для таких задач есть **двунаправленная приоритетная очередь (double-ended priority queue, DEPQ)**.

- DEPQ позволяет быстро извлекать **и минимум, и максимум**.
- Реализуется обычно через две кучи (мин-кучу и макс-кучу), либо через более сложные структуры (например, сбалансированные деревья).

В стандартных языковых библиотеках такой структуры обычно нет, её надо реализовывать самостоятельно.

2. Можно ли извлечь "другие" элементы кроме минимального или максимального?

- Нет, в приоритетной очереди нельзя быстро извлечь произвольный элемент — только корень кучи (минимум или максимум).
- Чтобы удалить произвольный элемент, придётся искать его и перестраивать структуру, что занимает $O(n)$ времени.
- Приоритетная очередь оптимизирована именно для быстрого доступа к элементу с наивысшим приоритетом.

3. Работает ли приоритетная очередь по принципу FIFO?

- **Нет!**
- FIFO (First In First Out) — это принцип работы обычной очереди: элементы выходят в том порядке, в котором вошли.
- В приоритетной очереди порядок извлечения определяется **приоритетом** элементов, а не временем их вставки.

Пример

В обычной очереди:

- Если вставить A, потом B, то при извлечении сначала выйдет A, потом B.

В приоритетной очереди:

- Если A имеет приоритет 2, а B — приоритет 1 (где меньший номер — выше приоритет), то при извлечении сначала выйдет B (приоритет 1), даже если вставлен после A.

Куча (англ. *heap*) в программировании — специализированная структура данных типа **дерева**, которая удовлетворяет *свойству кучи*: если *B* является узлом-потомком узла *A*, то $k(A) \geq k(B)$, где $k(X)$ — ключ (идентификатор) узла. Из этого следует, что элемент с наибольшим значением ключа всегда является корневым узлом кучи, поэтому иногда такие кучи называют *max-кучами* (в качестве альтернативы, если сравнение перевернуть, то наименьший элемент будет всегда корневым узлом, такие кучи называют *min-кучами*). Не существует никаких ограничений относительно того, сколько узлов-потомков

29. Преимущества и недостатки представления набора элементов в виде линейного списка vs массива

Что такое линейный список?

- В программировании чаще всего — **связный список** (linked list).
- Элементы (узлы) содержат данные и ссылку на следующий элемент (для однонаправленного списка). Может быть и двунаправленный список (каждый узел ссылается и на предыдущий).
- Элементы хранятся не подряд в памяти, а разбросаны, связаны указателями.

Что такое массив?

- Последовательный участок памяти, где элементы лежат подряд.
- Доступ по индексу — за константное время $O(1)$.

Сравнение: линейный список vs массив

Критерий	Массив	Связный список
Доступ по индексу	$O(1)$ — мгновенный, по индексу	$O(n)$ — нужно пройти по ссылкам
Вставка/удаление в середине	$O(n)$ — надо сдвигать элементы	$O(1)$ — просто переназначить ссылки
Вставка/удаление в начале	$O(n)$ — сдвиг элементов	$O(1)$ — быстрое изменение указателей
Вставка/удаление в конце	$O(1)$ — если заранее выделена память	$O(1)$ — если есть ссылка на конец
Память	Память под элементы подряд, экономно	Дополнительная память на ссылки
Кэш-производительность	Высокая — данные расположены подряд	Низкая — разбросаны по памяти

Примеры в JavaScript и Python

- В JS и Python нет "чистого" связного списка как встроенного типа, массивы — динамические и реализованы оптимально (JS Array, Python list).
- В Python есть модуль `collections.deque` — двунаправленная очередь, реализованная как связный список или кольцевой буфер.
- Для связных списков обычно пишут свою реализацию.

JS пример вставки в массив и связный список (упрощённо)

js

КопироватьРедактировать

```
// Вставка в массив (список)
```

```
let arr = [1, 2, 3];
```

```
arr.splice(1, 0, 10); // Вставить 10 на позицию 1 —  $O(n)$ 
```

```
// Связный список — вставка в середину  $O(1)$ , если есть узел
```

```
// но для поиска позиции нужно пройти —  $O(n)$ 
```

Вставка/удаление — $O(1)$, но с условием

- Да, сама операция вставки или удаления **узла**, если у вас уже есть ссылка на этот узел (или на соседний), происходит за **$O(1)$** – нужно лишь переназначить указатели (ссылки).
- Однако, **чтобы найти этот узел по значению или позиции**, если вы знаете только начало списка, нужно пройти по списку – это занимает **$O(n)$** .

Итог

Действие	Временная сложность
Поиск узла по индексу/значению	$O(n)$
Вставка или удаление при наличии узла	$O(1)$

Связные списки в языках со строгой типизацией

- **C++** – нет встроенного связанного списка, но в стандартной библиотеке STL есть `std::list`, который реализует двусвязный список.

cpp

КопироватьРедактировать

```
#include <list>
```

```
std::list<int> myList;
```

```
myList.push_back(10); // добавление в конец
```

```
myList.push_front(5); // добавление в начало
```

- **Java** – есть класс `LinkedList` в `java.util`, который реализует двусвязный список.

java

КопироватьРедактировать

```
import java.util.LinkedList;
```

```
LinkedList<Integer> list = new LinkedList<>();
```

```
list.add(10);
```

```
list.addFirst(5);
```

- **C#** — есть `LinkedList<T>` в пространстве имён `System.Collections.Generic`

30. Что означает «оценка с точностью до О-большое»? Для чего применяется?

Определение

О-большое (Big O notation) — это способ описать **асимптотическую сложность** алгоритма, то есть, как быстро растёт время работы или используемая память при увеличении размера входных данных.

Что означает?

- $O(f(n))$ — верхняя граница роста функции времени/памяти по размеру входа n .
- Например, если алгоритм работает за $O(n^2)$, значит время увеличивается примерно пропорционально квадрату размера входа.
- О-большое игнорирует константы и низкоуровневые детали, выделяя только основной вкладчик в рост.

Зачем это нужно?

- Позволяет сравнивать эффективность алгоритмов.
- Помогает выбирать более подходящий алгоритм для больших данных.
- Упрощает анализ, абстрагируясь от деталей реализации.

Пример в JS и Python

- Например, поиск элемента в массиве без сортировки:

js

КопироватьРедактировать

```
// Линейный поиск - O(n)
function linearSearch(arr, target) {
  for(let i = 0; i < arr.length; i++) {
    if(arr[i] === target) return i;
  }
  return -1;
}
```

- Быстрый поиск в отсортированном массиве (двоичный) — $O(\log n)$:

python

КопироватьРедактировать

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Обозначение:

$$f(n) = O(g(n))$$

означает, что функция $f(n)$ ограничена сверху функцией $g(n)$ с точностью до константы и для достаточно больших n . Формально:

$$\exists c > 0, n_0 > 0 : \quad \forall n \geq n_0, \quad f(n) \leq c \cdot g(n)$$

Для $O(n)$ это значит:

Если время работы алгоритма $T(n)$ — это $O(n)$, значит существует константа c , такая что при больших n :

$$T(n) \leq c \times n$$

Иными словами, время работы растёт не быстрее линейно с размером входа n .

Если предел существует и конечен:

$$\lim_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} = L < \infty$$

то $f(n) = O(g(n))$.

Вопрос 1

Здравствуйте! Меня зовут Владимир Дроздов, я поступаю в магистратуру Южного федерального университета на программу «Мобильная и веб-разработка» (09.04.04 Программная инженерия).

Я выбрал эту программу, потому что она напрямую соответствует моим профессиональным целям и интересам. Я хочу углубить навыки в веб-разработке, особенно во фронтенде, но также продолжить развитие в бэкенде (Node.js, PHP, Python). У меня есть опыт разработки, полученный в университете, на курсах, в ходе самостоятельного обучения, а также во время стажировки в РосАтом. В моём портфолио — собственные проекты, включая комплексную разработку веб-приложений.

Особое место среди моих работ занимает дипломная работа, посвященная созданию собственного инструмента для автоматического выявления уязвимостей во фронтенд-части веб-приложений. Он определяет XSS, SQL-инъекции, CSRF и другие уязвимости, анализируя HTML-код, DOM-структуру и сетевые запросы. Этот проект я хочу развивать и использовать как основу для

стартапа в сфере безопасности веб-приложений. Для этого мне необходимо расширить его функциональность, улучшить архитектуру и интегрировать современные технологии, в том числе мобильные и облачные решения, углубить свои знания в web-разработке и мобильной разработке.

Я выбрал именно Программу «Мобильная и веб-разработка» в ЮФУ, потому что она поможет мне реализовать мои цели: углубить технические знания, освоить новые инструменты, познакомиться с подходами к построению масштабируемых и безопасных приложений, что напрямую поможет вывести мой проект на новый уровень. Я вижу в обучении не только способ получения знаний, но и инструмент для превращения моего проекта в конкурентоспособный продукт, востребованный на рынке.

Я ожидаю, что к концу обучения я буду глубоко знать Python, Java, JavaScript, PHP, фреймворков Flask и Django, смогу улучшить свою научно-исследовательскую работу, а именно я планирую научить свою программу обрабатывать динамически загружаемый контент, расширить на веб-ресурсы, использующие фреймворки, расширить спектр обнаруживаемых уязвимостей.

Я считаю что готов к эффективному обучению, т. к. я обладаю прочной технической базой, опытом работы с веб-технологиями и чётким планом применения полученных знаний для создания инновационного решения в области информационной безопасности.

Я уверен, что смогу успешно освоить программу и применить полученные знания в будущей профессии.